МТУСИ

Обнаружение свободных парковочных мест

Выполнил студент группы БВТ2305
Кесслер А. С.

10.10.2025 г.

Введение

Рост числа автомобилей и нехватка парковок создают серьёзные городские проблемы — пробки, лишние выбросы и потерю времени.

Современные технологии компьютерного зрения и нейросети, такие как **YOLOv8** или **YOLOv11** позволяют автоматически определять свободные парковочные места в реальном времени, повышая эффективность городской инфраструктуры и комфорт водителей

Цели

Для достижения цели были поставлены следующие задачи:

1. Собрать и разметить датасет изображений с парковками.
2. Обучить модели YOLOv8 и YOLOv11 для детектирования автомобилей и выбрать лучшую
3. Реализовать программу для обработки видео в реальном времени и визуализации результатов детекции.
4. Разработать алгоритм определения занятости парковочных мест на основе координат обнаруженных объектов.
5. Реализовать оповещение пользователя через Telegram-бот о появлении свободного парковочного места.

Детектирование транспортных средств

Детектирование транспортных средств осуществляется с помощью моделей компьютерного зрения, обученных находить объекты на изображении.

Основные шаги:

- Предобработка— изменение размера изображений, нормализация.
- Модель детекции — используются нейросети, например:
  1. **YOLOv8** — современный быстрый одностадийный детектор;
  2. **YOLOv11** — более новая версия с улучшенной точностью и скоростью;
- Результат — координаты рамки (bounding box) и класс объекта (тип транспорта).

Детектирование транспортных средств

Модель	Достоинства	Недостатки
YOLOv8	Быстрая и лёгкая Хорошо работает в реальном времени Простая интеграция	Чуть ниже точность на сложных сценах
YOLOv11	Улучшенная архитектура Высокая точность Подходит для сложных условий	Требует больше ресурсов Новая, меньше исследований

Датасет

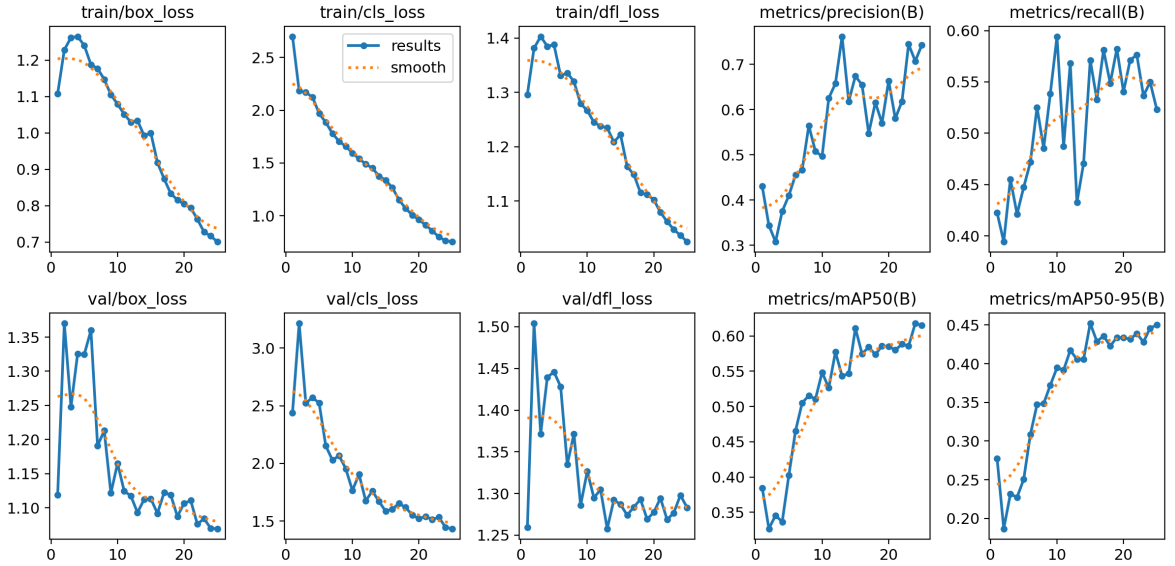
VehicleDetection
→ машины, мотоциклы, скорая помощь, грузовики, автобусы



YOLOv8

Метрики обучения/валидации

Эти графики показывают, как менялись значения потерь и метрик на тренировочной и валидационной выборках в течение 25 эпох. Основные категории: box loss (границы объектов), classification loss (классификация), DFL loss (распределённый фокальный лосс), и метрики качества (precision, recall, mAP).



Потери (Losses)

- **Train/Box, Cls, DFL Loss:** Стабильно снижаются, что говорит о корректной настройке весов и эффективном обучении.
- **Val/Box, Cls, DFL Loss:** Также уменьшаются, без признаков переобучения. Небольшие колебания в начале характерны для процесса стабилизации модели.

Метрики качества

- **Precision (точность):** Уверенно растёт, достигая около 0.7, что означает снижение числа ложных срабатываний.
- **Recall (полнота):** величивается до 0.55–0.6, показывая, что модель находит всё больше объектов. mAP50 и mAP50-95 — оба показателя растут на протяжении обучения; особенно mAP50-95 (около 0.45), что демонстрирует общее улучшение точности детекции при разных порогах IoU.

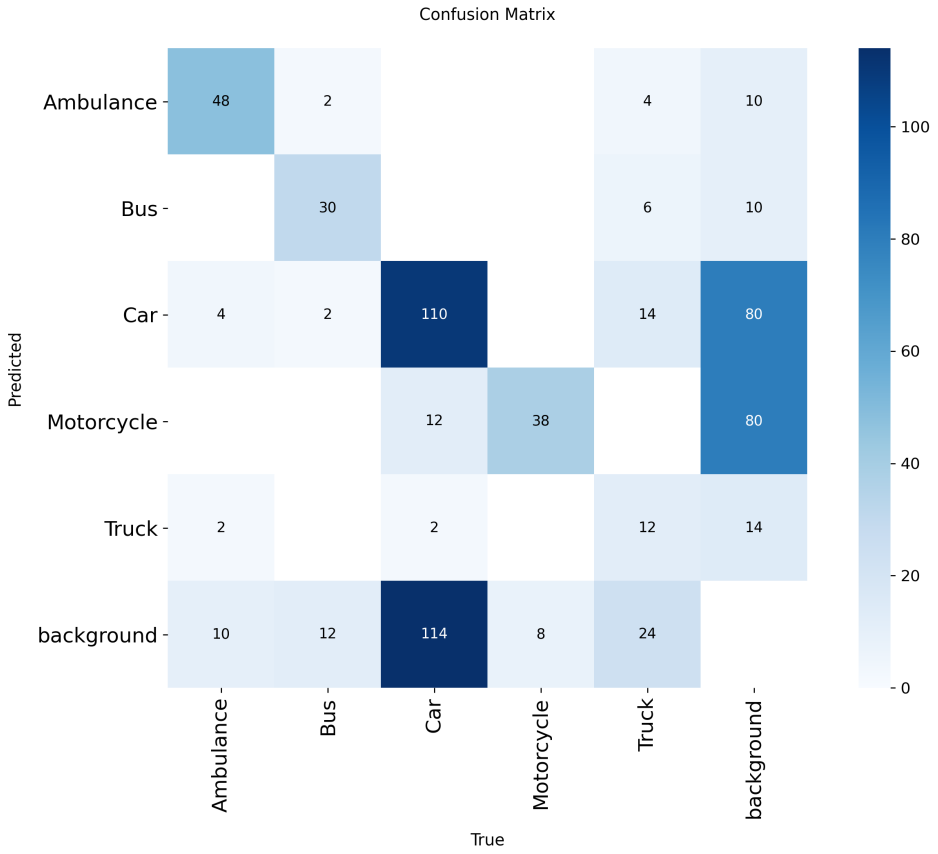
YOLOv8

Нормализованная матрица ошибок (Confusion Matrix)

Это таблица, показывающая, как часто модель путает один класс с другим. Значения нормализованы по строкам (предсказанные классы).

Модель показывает хорошие результаты при распознавании **легковых автомобилей (Car)** — это наиболее точно определяемый класс (110 корректных предсказаний).
Однако заметны проблемы с другими типами транспорта:

- **Bus** и **Truck** часто путаются с другими категориями, особенно с **Car** и **background**. Это может быть связано с визуальной схожестью форм и размеров на изображениях.
- **Motorcycle** частично ошибочно классифицируется как **background**, что указывает на трудности в обнаружении мелких объектов.
- **Ambulance** определяется относительно стабильно, но иногда воспринимается как **Truck** или **background**, что также может быть вызвано ограниченным количеством примеров в обучающем наборе.



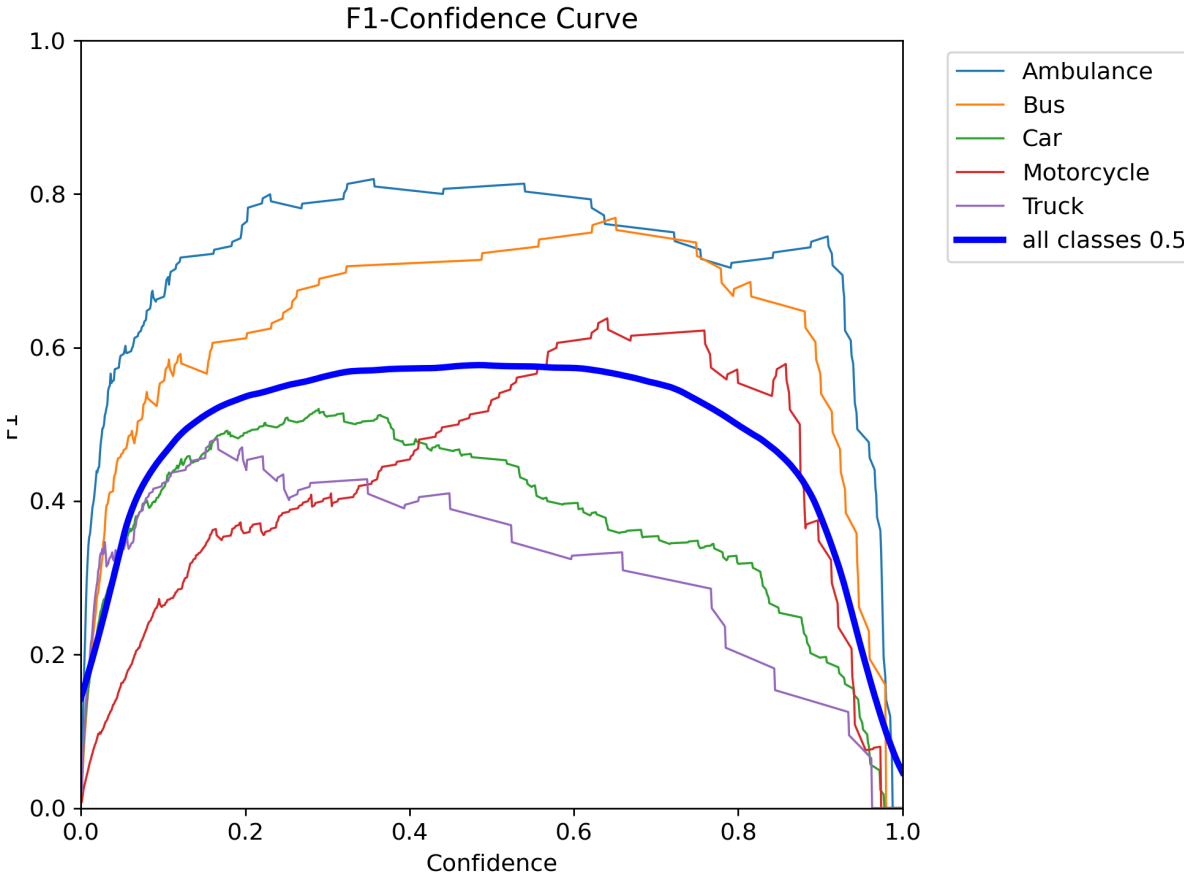
YOLOv8

F1-Confidence Curve(Кривая зависимости F1 от Confidence Threshold)

F1-мера — баланс между точностью и полнотой. Этот график показывает, как F1 зависит от порога уверенности, выше которого модель делает предсказание.

График **F1-Confidence Curve** показывает зависимость качества модели (F1-метрики) от порога уверенности предсказаний:

- **Толстая синяя линия** отображает усреднённый F1 по всем классам — наилучший результат достигается при уверенности около **0.48**, где **F1 ≈ 0.58**.
- Среди отдельных классов наилучшие результаты показывают **Ambulance** и **Bus**, их F1 превышает 0.8 на оптимальном интервале порога.
- **Car**, **Motorcycle** и **Truck** демонстрируют более низкие значения F1 (в пределах 0.4–0.5), что указывает на трудности в точной классификации этих объектов.



YOLOv8

Итог

Аспект	Состояние
Обучение	Стабильное, без переобучения
Потери	Снижаются адекватно
Точность (Precision)	Уверенный рост, достигает около **0.75**
Полнота (Recall)	Растёт до **0.55–0.6**, стабильное поведение
mAP	Около **0.55–0.6**, демонстрирует уверенный рост
Ошибки по классам	Модель сильно путает **bus** и **truck** с другими ТС; **car** и **ambulance** определяются точно
F1 и Confidence	Наилучший средний F1 ≈ **0.58** при пороге уверенности **0.48**



YOLOv11

Метрики обучения/валидации

Устойчивое снижение всех loss-функций (train/val): **box_loss**, **cls_loss**, **dfl_loss** плавно уменьшаются без признаков переобучения — тренд стабилен, кривые валидации повторяют форму обучающих

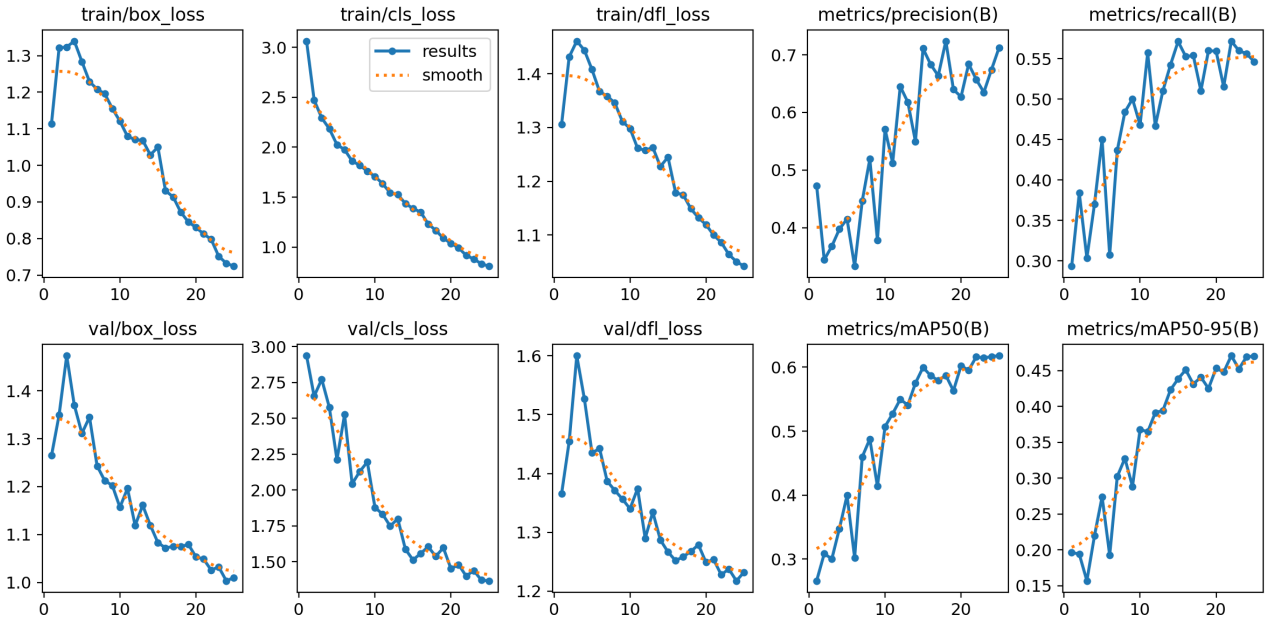
.

Высокие значения метрик:

- **Precision(B)** ≈ 0.70
- **Recall(B)** ≈ 0.55
- **mAP@0.5(B)** ≈ 0.60
- **mAP@0.5–0.95(B)** ≈ 0.45

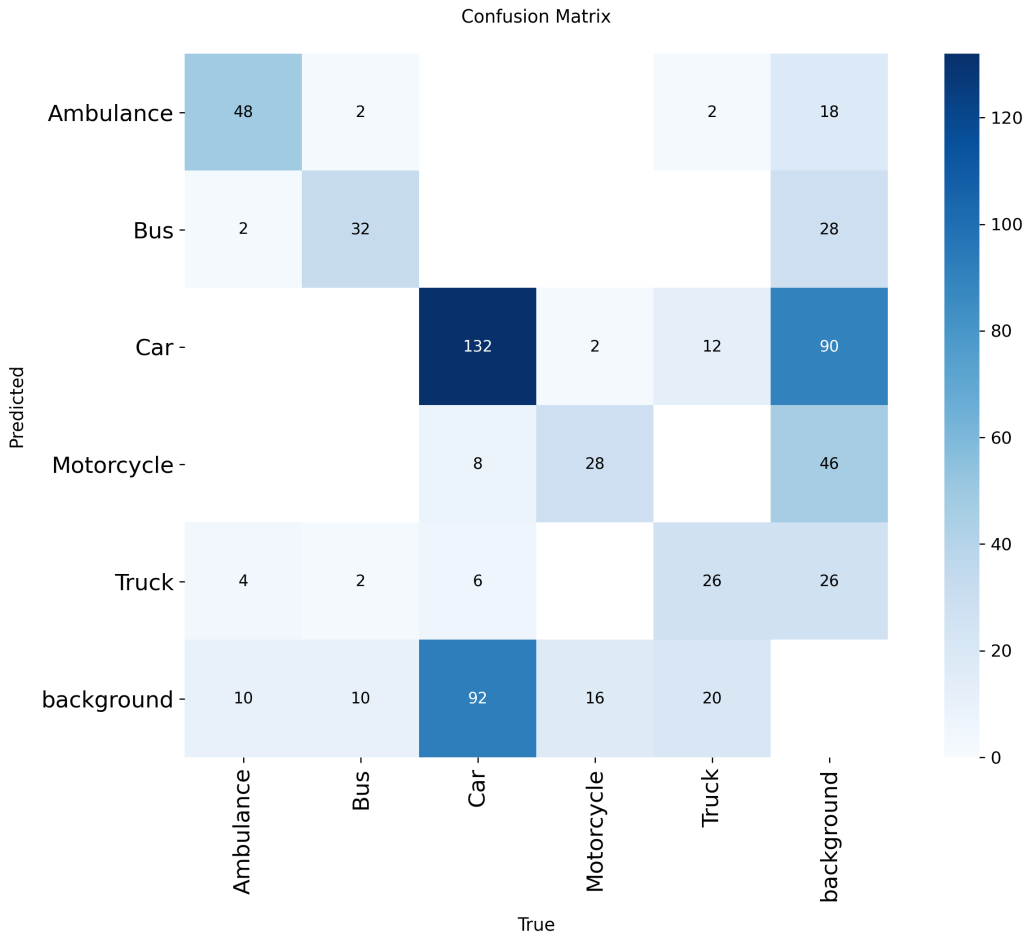
Стабильность на валидации: наблюдаются небольшие флуктуации метрик, что типично для детекции с умеренной сложностью классов и ограниченным числом примеров.

Модель обучена корректно, демонстрирует уверенный рост точности и адекватное обобщение. Потенциал для дальнейшего улучшения — лёгкая донастройка гиперпараметров (lr, augmentations) или увеличение размера датасета.



YOLOv11

Нормализованная матрица ошибок (Confusion Matrix)



Успехи:

- **Ambulance:** высокая точность (около 85–90%), незначительное пересечение с классом *background*.
- **Bus:** ≈90% точность, ошибки минимальны, модель уверенно различает автобус.
- **Car:** уверенное распознавание (132 верных из ≈236), но есть смешения с *background* и *truck*.
- **Motorcycle:** адекватные результаты (28 верных, часть спутана с *background* и *truck*).

Проблемные зоны:

- **Truck:** выраженное смешение — до 40% объектов ошибочно определяются как *car* или *background*.
- **Car:** частые пересечения с *truck* и *background*, что указывает на недостаточное разделение признаков по размерам и формам транспортных средств.
- **Background:** около 4–6% ложных срабатываний, чаще всего — на автомобили.

В целом — модель уверенно работает на большинстве классов, но требует балансировки и дообучения на трудных парах.

YOLOv11

F1-Confidence Curve(Кривая зависимости F1 от Confidence Threshold)

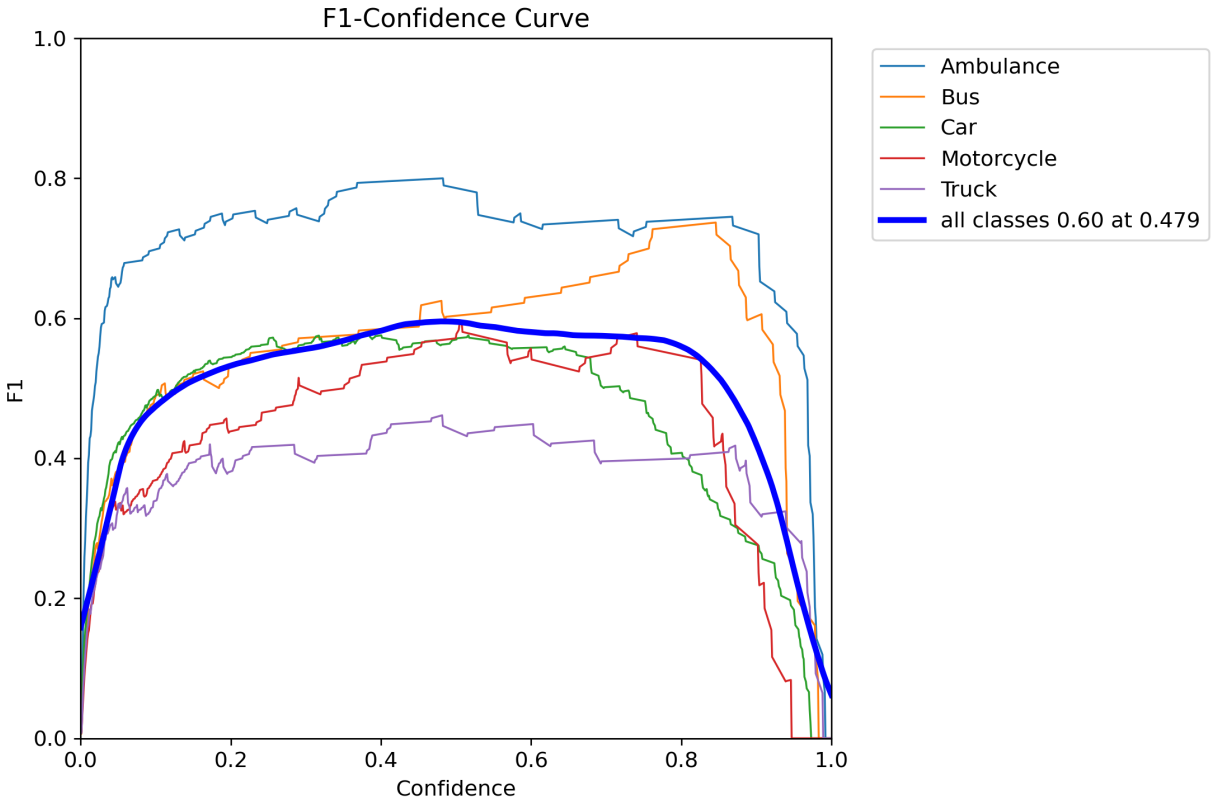
Поведение по классам:

- **Ambulance** и **Bus** демонстрируют стабильные F1-показатели даже при высоких confidence-порогах (до 0.9) — уверенная классификация с высокой точностью и полнотой.
- **Car** и **Truck** имеют выраженное падение F1 при confidence > 0.8, что говорит о неуверенности модели в различении этих классов.
- **Motorcycle** показывает умеренную стабильность, но общие значения F1 ниже среднего.

Общая кривая:

- Максимальное значение F1 ≈ 0.60 достигается при **confidence ≈ 0.48**, что указывает на оптимальный порог для детекции.
- При низких confidence (0.1–0.2) модель достигает **recall ≈ 0.99**, что говорит о высокой чувствительности, но с риском ложных срабатываний.

Вывод:
Модель хорошо улавливает редкие и визуально уникальные классы (Ambulance, Bus), однако испытывает трудности с визуально похожими объектами (*Car* ↔ *Truck*).



YOLOv11

Итог

Показатель	Анализ / Значение
Box Loss (Train/Val)	Стабильное снижение на обучении и валидации, переобучения не наблюдается
Classification Loss	Незначительные колебания на валидации — возможна путаница близких классов (car ↔ truck)
Precision	Средняя ≈ 0.70
Recall	Доходит до ≈ 0.55 — умеренный охват
mAP@0.5	Около 0.60 — модель корректно фиксирует объекты при стандартном IoU
mAP@0.5–0.95	Около ≈ 0.45 — стабильная точность даже при строгих условиях
Основные ошибки	Путаница между классами **truck** ↔ **car**, частые ложные фоны для car
Recall vs Confidence	Резкий спад recall при confidence > 0.8 у классов car и truck
Confusion Matrix	Truck ошибается чаще всего (до 40% → car/background), car — около 20% смешений



Промежуточный вывод

Модель **YOLOv11** продемонстрировала более высокую устойчивость и качество детекции по сравнению с **YOLOv8**. В процессе обучения наблюдается стабильное снижение всех видов потерь (box, cls, dfl) без признаков переобучения. YOLOv11 показывает уверенный рост метрик точности (Precision ≈ 0.70) и полноты (Recall ≈ 0.55), а также улучшенные значения **mAP@0.5 ≈ 0.60**, что свидетельствует о более точной локализации объектов по сравнению с YOLOv8.

По кривой F1–Confidence YOLOv11 сохраняет стабильное качество даже при увеличении порога уверенности, достигая наилучшего среднего значения F1 ≈ 0.60 при confidence ≈ 0.48. Это указывает на хорошее соотношение между полнотой и точностью при реальных условиях применения. В отличие от YOLOv8, модель демонстрирует лучшую устойчивость метрик и меньше подвержена флуктуациям на валидации.
Основные ошибки связаны с визуально схожими классами — **truck ↔ car**, где часть объектов путается или определяется как фон.

При этом классы **bus** и **ambulance** распознаются наиболее уверенно, а их показатели F1 достигают 0.8 и выше.
В целом **YOLOv11** можно считать оптимальным решением для задач детекции транспортных средств: она демонстрирует сбалансированное обучение, высокие значения метрик и устойчивость к изменению порога уверенности. Для дальнейшего повышения точности рекомендуется:

- расширить обучающую выборку примерами классов *truck* и *bus* в сложных условиях (перекрытия, ночная съёмка, различные ракурсы);
- уточнить аннотации и провести балансировку классов для уменьшения перекрёстных ошибок;
- настроить индивидуальные пороги confidence по классам;
- применить **Test-Time Augmentation (TTA)** и **soft-NMS** для повышения устойчивости и улучшения результатов на реальных данных.

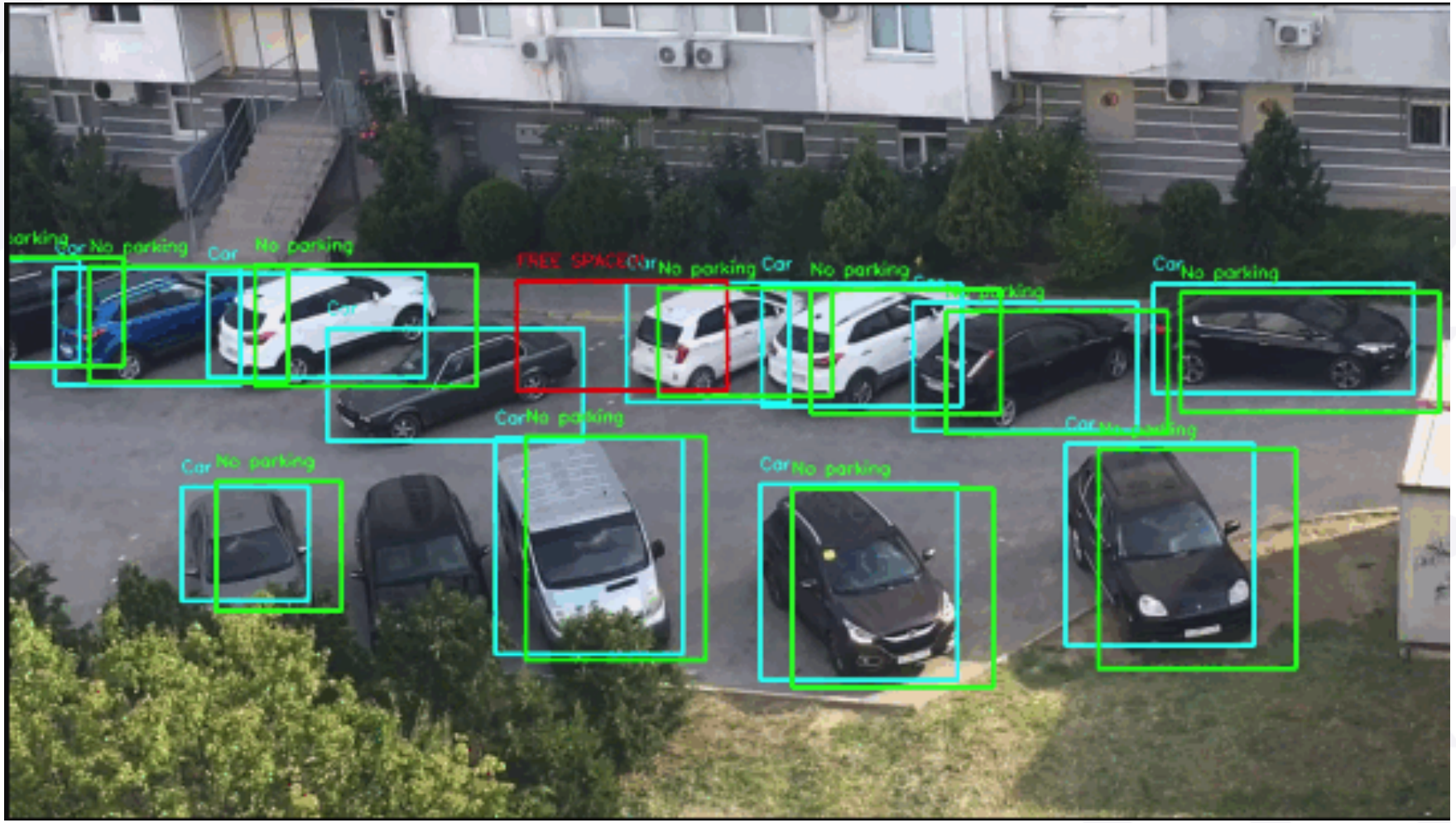
Написание кода

С моделью определились, теперь нужно написать программу которая будет детектировать освободившееся место на потоке видеокамеры и отправлять уведомление в телеграм бота. Код буду писать на Python.

Я реализовал следующие шаги
1) Подключаем камеру с помощью библиотеки CV. Разработку я делал на заранее записанном видео, но если работаем с веб-камерой, то необходимо просто передать cv2.VideoCapture() цифру ноль. Далее работаем с каждым кадром (берем каждый кадр видео и прогоняем его через нашу модель).
2) Следующий шаг: работа YOLO детектора. Берем только Bounding Boxes необходимых объектов класса car.
3) Теперь начинается творческая часть. Что мы подразумеваем под паковочными местами? Самое простое и логичное: взять места на которых стоят машины! То есть, под всеми машинами, которые определились в кадре, находятся парковочные места. Что ж, для начала подойдет такой подход (усложнить всегда успеем)
4) Теперь будем детектировать сами машины в кадре. Это уже динамическая часть отработки программы. Здесь и возникают основные сложности. Как определить, что машина стоит на паркоместе? Сравнить пересечение их BoundingBoxes, то есть нам нужно использовать Intersection over Union (IoU).
5) Теперь вопрос: как «вытащить» нужное IoU и сказать модели, что это именно наша машина? Сделаем несколько фильтров
6) И вот когда все три условия соблюдены на протяжении 10 кадров, мы наконец-то можем пометить выбранный BBox как свободное парковочное место и переключить флаг free_parking_space в положение True.
7) Стоит сделать обратную вещь: если free_parking_space=True, но парковочное место занимают, то у нас опять нет свободного места :(
8) Осталось совсем немного - прикрутить telegram сервис по информированию

Полностью код можно посмотреть в моём гитхабе:
https://github.com/LightninG8/MTUCI/blob/main/4%20%D0%A1%D0%B5%D0%BC/%D0%9F%D0%9F/main.py

Результат


Car No parking
Car No parking
FREE SPACE
Car No parking
Car No parking
Car No parking
Car No parking
Car No parking
Car No parking

Код

```python
import cv2
import numpy as np
import requests
from ultralytics import YOLO

# === Настройки Telegram ===
TOKEN = "7907985921:AAFAYj2Xv-qHxxH38z6loKltgPI0tY1ZM2o"
chat_id = "777795628"

def send_photo_file(chat_id, img_path):
    with open(img_path, 'rb') as f:
        files = {'photo': f}
        requests.post(f'https://api.telegram.org/bot{TOKEN}/sendPhoto?chat_id={chat_id}', files=files)

def send_telegram_message(message):
    requests.get(f'https://api.telegram.org/bot{TOKEN}/sendMessage?chat_id={chat_id}&text={message}').json()

# === Путь до видео ===
video_path = './test_5.mp4'

# === Функция для отрисовки Bounding Box ===
def draw_bbox(image, x, y, w, h, text, color=(0, 255, 0)):
    cv2.rectangle(image, (x, y), (x + w, y + h), color, 2)
    cv2.putText(image, text, (x, y - 10), cv2.FONT_HERSHEY_SIMPLEX, 0.4, color, 1)
    return image

# === Загружаем YOLOv8 ===
model = YOLO("yolov8n.pt")  # используем локальный файл

# === Переменные для парковки ===
first_frame_parking_spaces = None
free_parking_timer = 0
free_parking_timer_bag1 = 0
free_parking_space = False
free_parking_space_box = None
check_det_frame = None
telegram_message = False

# === Работа с видео ===
video_capture = cv2.VideoCapture(video_path)

while video_capture.isOpened():
    ret, frame = video_capture.read()
    if not ret:
        video_capture.set(cv2.CAP_PROP_POS_FRAMES, 0)
        continue

    height, width, _ = frame.shape

    # Прогон кадра через YOLOv8
    results = model(frame)[0]  # первый результат
    boxes = []
    scores = []
    # NMS для фильтрации дублирующих машин
    cars_boxes = []  # <--- добавляем эту строку
```

```python
    for r in results.boxes:
        cls_id = int(r.cls[0])
        score = float(r.conf[0])
        # 2 = car в COCO (проверяем)
        if cls_id == 2 and score > 0.25:
            x1, y1, x2, y2 = map(int, r.xyxy[0])
            boxes.append([x1, y1, x2 - x1, y2 - y1])
            scores.append(score)

    # === Первый кадр: определяем парковочные места ===
    if first_frame_parking_spaces is None:
        first_frame_parking_spaces = boxes.copy()
        first_frame_parking_scores = scores.copy()
        parking_spaces = boxes.copy()
    else:
        # NMS для фильтрации дублирующих [redacted]
        indices = cv2.dnn.NMSBoxes(boxes, scores, 0.0, 0.4)
        indices = indices.flatten() if len(indices) > 0 else []
        cars_boxes = [boxes[i] for i in indices]

        # === Проверка свободных мест через IoU ===
        if len(cars_boxes) > 0 and len(first_frame_parking_spaces) > 0:
            def calculate_iou(box, other_boxes):
                box_area = box[2] * box[3]
                other_areas = np.array([b[2]*b[3] for b in other_boxes])
                ious = []
                for b in other_boxes:
                    xA = max(box[0], b[0])
                    yA = max(box[1], b[1])
                    xB = min(box[0]+box[2], b[0]+b[2])
                    yB = min(box[1]+box[3], b[1]+b[3])
                    inter = max(0, xB-xA) * max(0, yB-yA)
                    iou = inter / (box_area + (b[2]*b[3]) - inter)
                    ious.append(iou)
                return ious

            for idx, space in enumerate(first_frame_parking_spaces):
                overlaps = calculate_iou(space, cars_boxes)
                max_iou = max(overlaps) if overlaps else 0

                if not free_parking_space and 0.0 < max_iou < 0.4:
                    free_parking_timer += 1
                    if free_parking_timer >= 10:
                        free_parking_space = True
                        free_parking_space_box = space
                elif free_parking_space:
                    overlaps = calculate_iou(free_parking_space_box, cars_boxes)
                    if overlaps and max(overlaps) > 0.6:
                        free_parking_space = False
                        telegram_message = False
```

```python
    # === Отрисовка парковочных мест ===
    for space in first_frame_parking_spaces:
        if free_parking_space and space == free_parking_space_box:
            frame = draw_bbox(frame, *space, "FREE SPACE!!!", (0, 0, 255))
            if not telegram_message:
                cv2.imwrite('./image_test_free.png', frame)
                send_telegram_message("Свободное место! Давай, жми скорее!!!")
                send_photo_file(chat_id, './image_test_free.png')
                telegram_message = True
        else:
            frame = draw_bbox(frame, *space, "No parking")

    # === Отрисовка машин ===
    for car in cars_boxes:
        frame = draw_bbox(frame, *car, "Car", (255, 255, 0))

    cv2.imshow("Parking Space", frame)
    if cv2.waitKey(1) & 0xFF == ord('q'):
        break

video_capture.release()
cv2.destroyAllWindows()
```

Вывод

- На основе готово датасета были обучены и сравнены две современные модели детекции — YOLOv8 и YOLOv11.
- По результатам анализа метрик (Precision, Recall, mAP, F1) и визуальной оценки детекций, YOLOv11 показала наилучшие результаты по точности и устойчивости. Модель уверенно распознаёт автомобили на парковке даже при частичных перекрытиях и меняющемся освещении, что делает её оптимальным выбором для решения поставленной задачи.
- На основе обученной модели была реализована программа для обработки видеопотока в реальном времени, которая выполняет детекцию транспортных средств, анализирует координаты объектов и определяет степень занятости парковочных мест.
- Дополнительно разработан Telegram-бот, обеспечивающий оперативное уведомление пользователей о появлении свободных парковочных мест, что делает систему удобной для практического применения.
- Таким образом, проект продемонстрировал возможность интеграции современных нейросетевых технологий в задачи интеллектуального мониторинга парковок. Реализованная система является основой для дальнейшего расширения — внедрения поддержки нескольких камер, прогнозирования занятости и интеграции с «умными» городскими системами.

Источники информации

1. Автоматизированное детектирование и классификация объектов в транспортном потоке на спутниковых снимках города - В.С. Тормозов (18 April 2024)
2. YOLOv11 for Vehicle Detection: Advancements, Performance, and Applications in Intelligent Transportation Systems - Mujadded Al Rabbani Alif (30 Oct 2024)
3. Vehicle Detection and Classification via YOLOv8 and Deep Belief Network over Aerial Image Sequences - Naif Al Mudawi (8 October 2023)
4. Fine-Grained Vehicle Classification in Urban Traffic Scenes using Deep Learning - Rana Hammad Raza (17 November 2021)
5. Vehicle detection and classification using an ensemble of EfficientDet and YOLOv8 - Ivan Miguel Pires (13 August 2024)